National School of Business Management

DS 201.3 - Introduction to Data Science

Coursework Report: FOC

Student Performance Classification using Machine Learning

Submitted by:

Vihasna, PJ - (38602) -> pjvihasna@students.nsbm.ac.lk

Submitted to:

Ms. Thilini Bakmeedeniya

Date of Submission:

25th of May 2026

Year 2 – Semester 1 ->Batch 25.2

Table of Content:

1. Introduction

- In this project, a machine learning model is developed to classify student performance based on various academic and behavioral factors.
- The **K-Nearest Neighbors (KNN)** algorithm is used to predict the final grade of students.
- This approach helps in identifying patterns in student data and improving academic decision-making.

2. Objectives

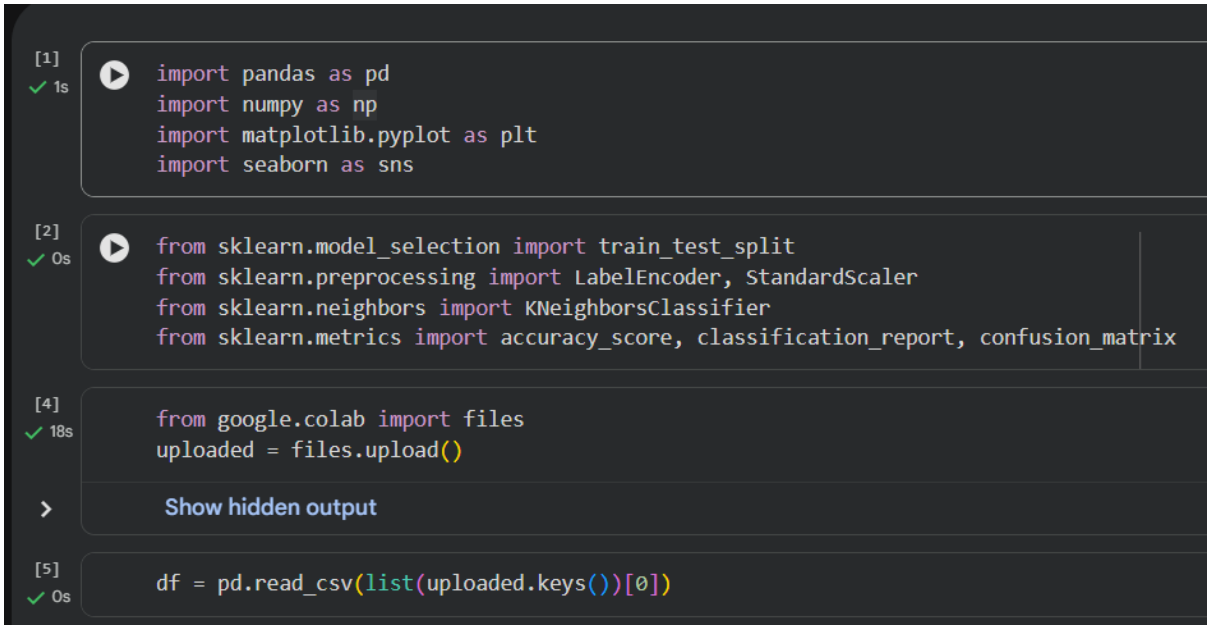
The main objectives of this project are:

- To analyze student performance data
- To preprocess and clean the dataset
- To build a classification model using KNN
- To evaluate the model performance using different metrics

3. Tools and Technologies

This project was implemented using:

- Python programming language
- Google Colab environment
- Libraries: pandas, numpy, matplotlib, seaborn, scikit-learn

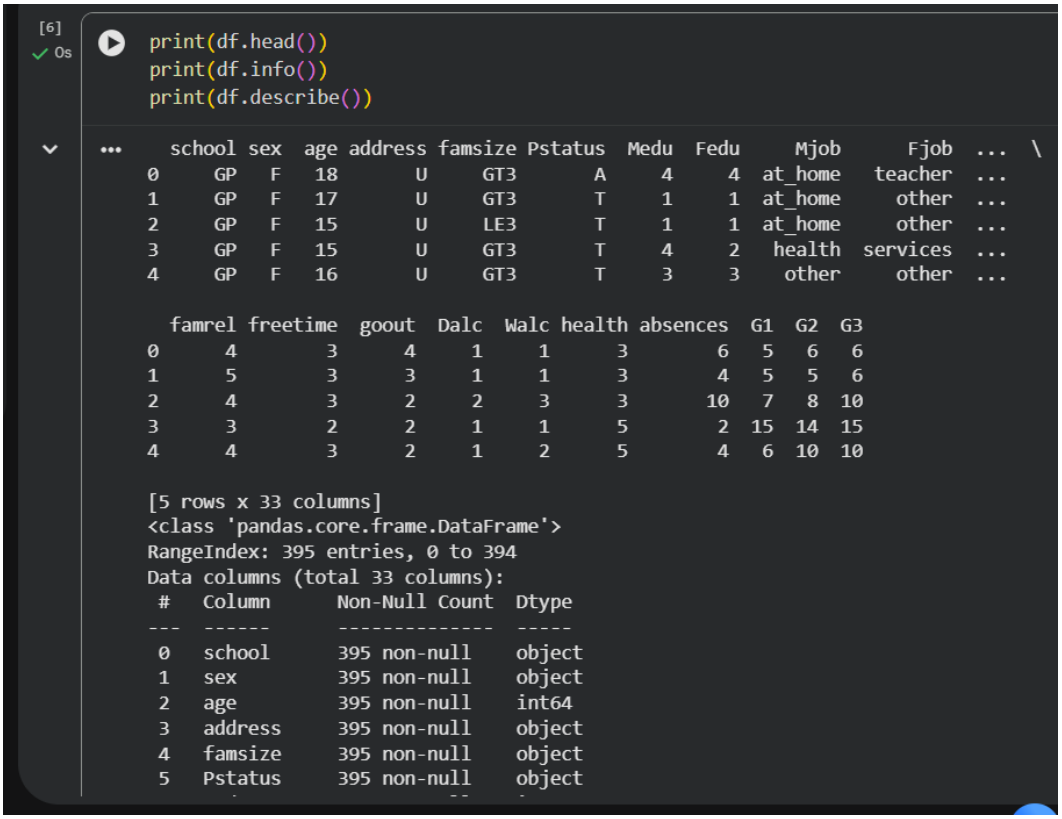


4. Dataset Description

The dataset contains multiple features related to student performance such as study habits, attendance, and stress levels. The target variable is **Grade**, which represents the student's final performance.

- **<u>Dataset Preview and Dataset Information</u>**

  ->The following screenshot shows the first few rows of the dataset using the `df.head()` function, which helps to understand the structure and columns of the dataset.



```
[6]
✓ 0s
print(df.head())
print(df.info())
print(df.describe())
```

```
  school sex  age address famsize Pstatus  Medu  Fedu     Mjob      Fjob  ... \
0     GP   F   18       U     GT3       A     4     4  at_home   teacher  ...
1     GP   F   17       U     GT3       T     1     1  at_home     other  ...
2     GP   F   15       U     LE3       T     1     1  at_home     other  ...
3     GP   F   15       U     GT3       T     4     2   health  services  ...
4     GP   F   16       U     GT3       T     3     3    other     other  ...

  famrel freetime  goout  Dalc  Walc health absences  G1  G2  G3
0      4        3      4     1     1      3        6   5   6   6
1      5        3      3     1     1      3        4   5   5   6
2      4        3      2     2     3      3       10   7   8  10
3      3        2      2     1     1      5        2  15  14  15
4      4        3      2     1     2      5        4   6  10  10

[5 rows x 33 columns]
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 395 entries, 0 to 394
Data columns (total 33 columns):
 #   Column      Non-Null Count  Dtype
---  ------      --------------  -----
 0   school      395 non-null    object
 1   sex         395 non-null    object
 2   age         395 non-null    int64
 3   address     395 non-null    object
 4   famsize     395 non-null    object
 5   Pstatus     395 non-null    object
```

->This figure displays the dataset structure, including column names, data types, and non-null values obtained using `df.info()`.

```
 5   Pstatus     395 non-null    object
 6   Medu        395 non-null    int64
 7   Fedu        395 non-null    int64
 8   Mjob        395 non-null    object
 9   Fjob        395 non-null    object
 10  reason      395 non-null    object
 11  guardian    395 non-null    object
 12  traveltime  395 non-null    int64
 13  studytime   395 non-null    int64
 14  failures    395 non-null    int64
 15  schoolsup   395 non-null    object
 16  famsup      395 non-null    object
 17  paid        395 non-null    object
 18  activities  395 non-null    object
 19  nursery     395 non-null    object
 20  higher      395 non-null    object
 21  internet    395 non-null    object
 22  romantic    395 non-null    object
 23  famrel      395 non-null    int64
 24  freetime    395 non-null    int64
 25  goout       395 non-null    int64
 26  Dalc        395 non-null    int64
 27  Walc        395 non-null    int64
 28  health      395 non-null    int64
 29  absences    395 non-null    int64
```

```
dtypes: int64(16), object(17)
memory usage: 102.0+ KB
None
              age        Medu        Fedu  traveltime   studytime    failures  \
count  395.000000  395.000000  395.000000  395.000000  395.000000  395.000000
mean    16.696203    2.749367    2.521519    1.448101    2.035443    0.334177
std      1.276043    1.094735    1.088201    0.697505    0.839240    0.743651
min     15.000000    0.000000    0.000000    1.000000    1.000000    0.000000
25%     16.000000    2.000000    2.000000    1.000000    1.000000    0.000000
50%     17.000000    3.000000    2.000000    1.000000    2.000000    0.000000
75%     18.000000    4.000000    3.000000    2.000000    2.000000    0.000000
max     22.000000    4.000000    4.000000    4.000000    4.000000    3.000000

            famrel    freetime       goout        Dalc        Walc      health  \
count  395.000000  395.000000  395.000000  395.000000  395.000000  395.000000
mean     3.944304    3.235443    3.108861    1.481013    2.291139    3.554430
std      0.896659    0.998862    1.113278    0.890741    1.287897    1.390303
min      1.000000    1.000000    1.000000    1.000000    1.000000    1.000000
25%      4.000000    3.000000    2.000000    1.000000    1.000000    3.000000
50%      4.000000    3.000000    3.000000    1.000000    2.000000    4.000000
75%      5.000000    4.000000    4.000000    2.000000    3.000000    5.000000
max      5.000000    5.000000    5.000000    5.000000    5.000000    5.000000

          absences          G1          G2          G3
count  395.000000  395.000000  395.000000  395.000000
mean     5.708861   10.908861   10.713924   10.415190
std      8.003096    3.319195    3.761505    4.581443
min      0.000000    3.000000    0.000000    0.000000
25%      0.000000    8.000000    9.000000    8.000000
50%      4.000000   11.000000   11.000000   11.000000
75%      8.000000   13.000000   13.000000   14.000000
max     75.000000   19.000000   19.000000   20.000000
```

5. Data Preprocessing

5.1 Handling Missing Values

- Missing values in the dataset were handled using the forward fill method to ensure there are no empty cells affecting the model performance.

Missing Values Check

The following screenshot shows the count of missing values in each column before handling them.

```
print(df.isnull().sum())
```

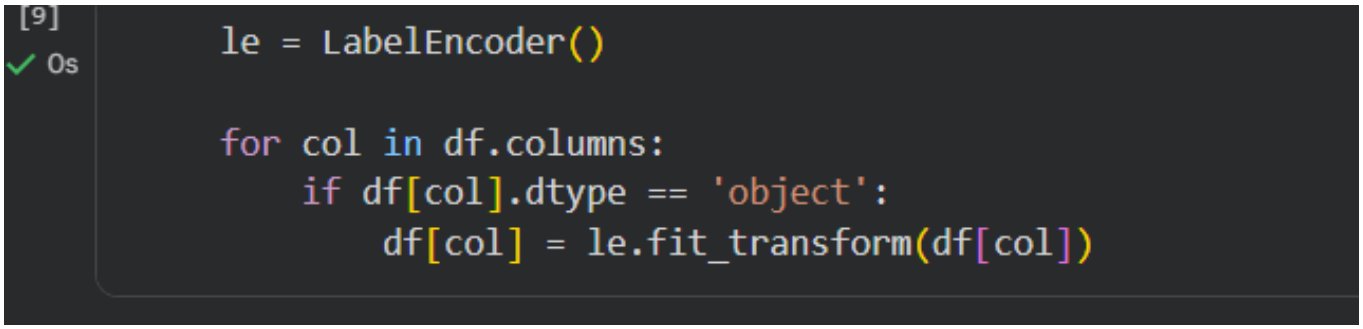
```
school        0
sex           0
age           0
address       0
famsize       0
Pstatus       0
Medu          0
Fedu          0
Mjob          0
Fjob          0
reason        0
guardian      0
traveltime    0
studytime     0
failures      0
schoolsup     0
famsup        0
paid          0
activities    0
nursery       0
higher        0
internet      0
romantic      0
famrel        0
freetime      0
goout         0
Dalc          0
Walc          0
health        0
absences      0
G1            0
G2            0
G3            0
dtype: int64
```

5.2 Encoding Categorical Data

Categorical variables were converted into numerical format using Label Encoding so that they can be processed by the machine learning model.

Encoded Dataset

This figure shows the dataset after converting categorical values into numeric values.



5.3 Feature Selection

The dataset was divided into input features (X) and the target variable (y). The **Grade** column was selected as the target.

5.4 Splitting the Dataset

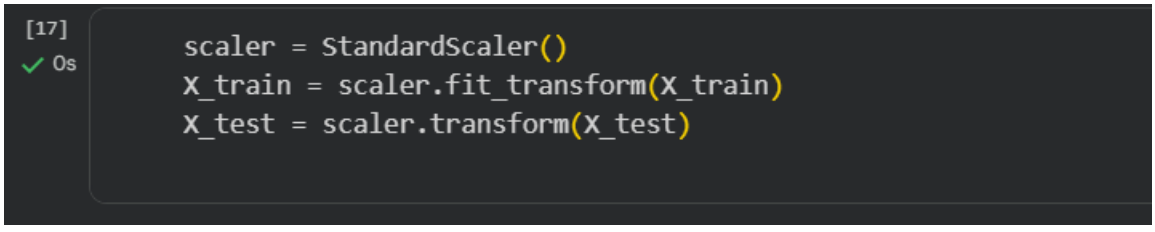
The dataset was split into training and testing sets using an 80:20 ratio. This allows the model to learn from training data and be evaluated on unseen test data.

5.5 Feature Scaling

Feature scaling was applied using StandardScaler to normalize the data and improve the performance of the KNN algorithm.

Scaled Features

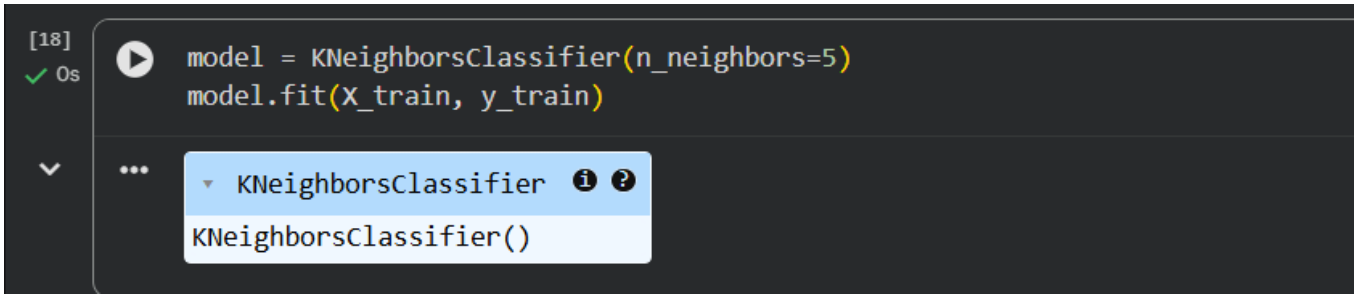
The following screenshot shows the transformed feature values after applying scaling.



6. Model Building

- The K-Nearest Neighbors (KNN) model was built using 5 nearest neighbors.

- This means the model classifies a data point based on the majority class among its 5 closest neighbors.



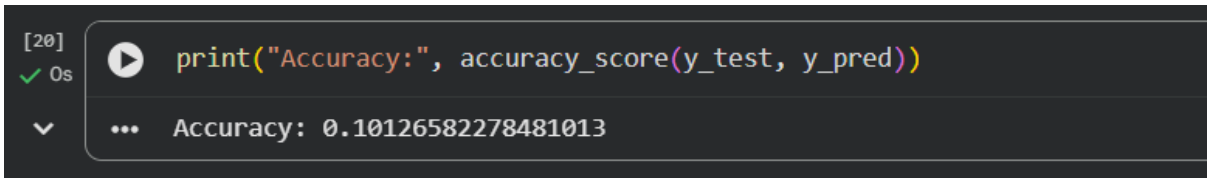
7. Model Evaluation

7.1 Accuracy

Accuracy is used to measure how well the model performs in predicting the correct output.

Model Accuracy

The screenshot below shows the accuracy score of the KNN model after testing.



7.2 Classification Report

The classification report provides detailed performance metrics such as precision, recall, and F1-score for each class.

Classification Report

This figure shows the classification report generated for the model predictions.

```
[21]
print(classification_report(y_test, y_pred))

              precision    recall  f1-score   support

           0       0.11      0.40      0.17         5
           5       0.00      0.00      0.00         4
           6       0.00      0.00      0.00         6
           7       0.50      1.00      0.67         1
           8       0.17      0.17      0.17         6
           9       0.14      0.20      0.17         5
          10       0.08      0.09      0.08        11
          11       0.00      0.00      0.00         5
          12       0.25      0.20      0.22         5
          13       0.00      0.00      0.00         5
          14       0.00      0.00      0.00         6
          15       0.00      0.00      0.00        10
          16       0.00      0.00      0.00         4
          17       0.00      0.00      0.00         3
          18       0.20      1.00      0.33         1
          19       0.00      0.00      0.00         2

    accuracy                           0.10        79
   macro avg       0.09      0.19      0.11        79
weighted avg       0.06      0.10      0.07        79

/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predi
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predi
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predi
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
```

7.3 Confusion Matrix

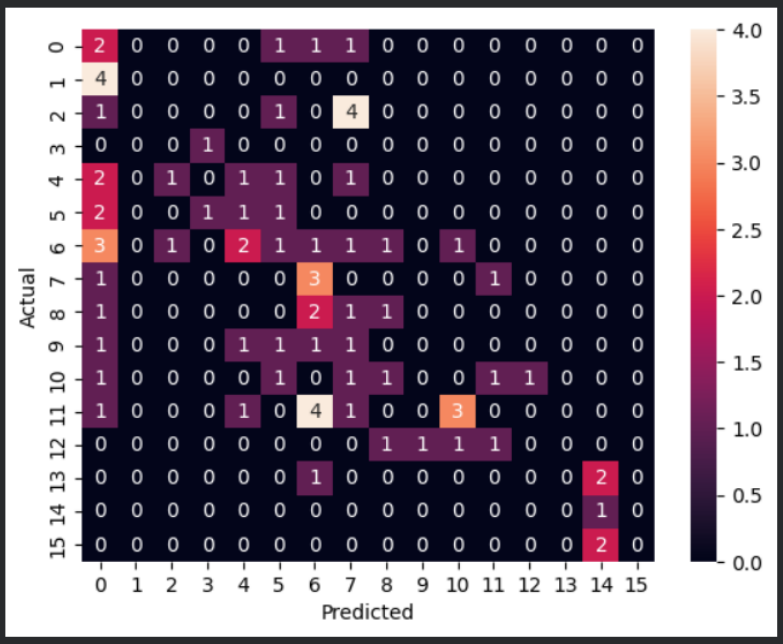
A confusion matrix is used to visualize the performance of the classification model by comparing actual and predicted values.

Confusion Matrix

The following heatmap represents the confusion matrix of the model.

[23]
✓ 0s

```
cm = confusion_matrix(y_test, y_pred)
sns.heatmap(cm, annot=True, fmt='d')
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
```



8. Results and Discussion

- The KNN model showed satisfactory performance in classifying student grades.
- The use of feature scaling improved the model accuracy.
- However, the performance can vary depending on the value of K and the quality of the dataset.

9. Conclusion

- In conclusion, the KNN algorithm was successfully applied to classify student performance.
- The model was able to learn patterns from the dataset and provide accurate predictions.
- Proper data preprocessing played a key role in achieving better results.

10. Future Improvements

- Test different values of K (e.g., 3, 7, 9)
- Use advanced models such as Decision Tree or Random Forest
- Perform hyperparameter tuning
- Use a larger and more detailed dataset

11. References

- Scikit-learn documentation

- Python official documentation